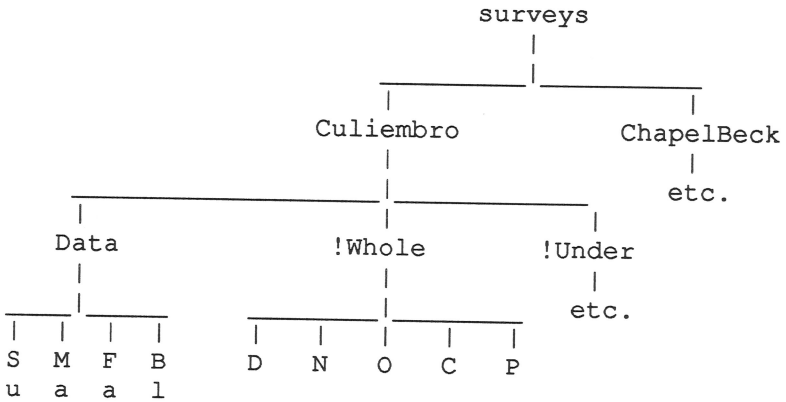
> BBC:$.Survey.!ArchySU.!ReadMe

This is an outline of an idea for thoughts about the possible "look and feel" of SU-BBC if it going to work under Archimedes DeskTop and be interruptible, be able to live on the icon bar, be started by dragging survey file icons into it etc..

What will it look like ?

There will be an application with the imaginative name of !Survey and some suitable Icon (ho, ho - this could be interesting ?). Right now, this is living in BBC:$.Survey.!SU. You will need a separate directory somewhere (this could be exactly where it is now, for all the difference it makes) with all your survey data. Now when this directory (lets call it "Surveys") is open, you will have a subdirectory for each survey area (just like the current version). If you open a survey directory, you want to be able to pick up an icon from it and drag it onto the survey icon on the icon bar, and for everything else to happen from then on. Now this ISN'T possible with your curent survey directory structure, since a survey may have more than one master D-file (which is the one you want to drag) which are hidden in the D subdirectory. Obviously we need to change the directory structure to make the dragging of icons into survey programs easier. Try this :



Now Culiembro.!Whole.D would contain the data file which now lives in Culiembro.D.CuliembroA, ie. the control file. You would proceed by dragging the !Whole directory into the Survey icon, whereupon it would calculate the survey, building the .N, .O, .C and .P files as usual. Survey would get an environment string with "...:$....Surveys.Culiembro.!Whole" in it, so would strip off the last bit of path and append the .Data directory to it to find the data files.

OK, that was a first idea. Unfortunately it doesn't quite meet the criterion we want, namely that it is easy to drag a survey data file into Twin/Edit. You can do this for all the files except the controlling .D file, which is hidden in a !directory. This is basically caused by the Filer : you can't have an icon to represent a directory unless it starts with a "!", but then you can't open a viewer quite so easily. What we really need are two sorts of thing that have directory icons, one is the obvious case of an application, for which it is quite reasonable not to open the directory easily, but the other is the case of a structured data directory which we want to have an icon we can recognise and drag about, but which we can open to edit the data and look at the print file. Also, double clicking mustn't try to run it !

Tragically, although we have the source to RISCOS Filer, we can't really go around supplying modified versions of it to make life easier because Acorn would (quite reasonably) object rather strongly. Anyway, we wouldn't really want software which was only useable on a non-standard system, no matter how traditional this has become !

If we modified the above scheme so that the directory was called "Whole" in a fully pling-free manner, all we would lose was the ability to associate a sprite with a survey. This is a shame, but I can't see an easy way round it at present.

We would like to have the program spot it if you dragged the .D file into it as well, and treat it as if you had dragged the directory. This shouldn't be especially difficult.

```
**************************************************************************
**                                                                      **
** The obvious question is, could we provide a patch-on bit of code that **
** would make the filer behave the way we want without arousing Acorn's  **
** justified anger ? THE ONLY change we want is to have the filer look   **
** for iconsprites in every directory it opens, not just applications.   **
** In every other way (ie. in respect of double clicks) we want our      **
** directories to behave the way they do now, so we can open them and    **
** play inside. I mean - I just hate the directory icon anyway !         **
**                                                                      **
**************************************************************************
```

If I was writing RISCOS, I would have space in the directory not just for a creation date, but also for a type. It is silly relying on part of the name to tell you that something is an application - names are too short already without losing a character. The point being that if directories had types like what files do, then you could make the system extensible in much the same way - most directories would be the default type and would get the default icon, others would be applications and would get an icon from inside, while others could be types defined by Acorn/SoftHouses/Users which would have a directory$type_xxx variable and an associated icon, and yet others would be non-application types which nevertheless had their own icon in a !Sprites file inside. All this seems pretty logical, given the way files work, so why didn't Acorn do it that way ?
OK, so that wouldn't, in fact, do any good at all for directories on NET: or BBC:, but that didn't stop them providing dates, did it ?

> BBC:$.Survey.!SU.!CloseNote

See articles by Patrick Warren in Cambridge Underground for full background to the closure alogorithm. Release 2 so far calculates the standard error for each survey leg (note that space has an anisotropy caused by the choice of axes for this sort of computation, but for 99.9% of normal surveys, this is not a problem).

This is slightly horrible : the dX%, dY%, dZ% are variances, which gives them the dimensions of length squared and makes them impossible to store usefully under the decamicron convention. There are two solutions, one uses more memory, one uses more processing power.

1) store as five-byte reals
2) square-root them and store as decamicron standard errors, squaring them back up later for summation.

(Just as an experiment, try square-rooting and resquaring 3000 numbers on the ARM in BASIC and time it).

A more radical solution, eating lots of memory and patently inefficient on the 32-bit ARM, would be to store everything as five-byte reals. One problem here is that you couldn't read files back in this format with a compiled language with 32 or 64 bit reals. Hence I don't like it. All the files should have 32-bit decamicron lengths.

OK, OK, OK, its time to consider the radical redesign of the filing space : we currently use the vector file for display graphics, but actually only use a tiny fraction of that information, ie. the internal numbers from/to. We should really design things so that from/to live in one area (which we can file), vectors live in another, and standard errors live in a third. This means we could lob the standard error area back on a heap (or reuse it for coordinates), and use less disc for our graphics data.

Along with this, we should implement procedures to access elements in these areas so that indirections don't appear in the higher level code.

This is called data abstraction and should be practiced regularly.

This doesn't seem to contain much of the original discussion on the implementation of the Warren algorithm does it ? This is because you worked it out on a train without being able to type it in. Hence it was all carved on stone tablets with a flint ball-point. IF you manage to find the note-book again, you could convert the outline functional spec. into something like a how-to-do-it outline. I seem to remember the data structures causing some bogglement, to the extent of committing the eternal heresy of thinking "This would be easier in a language with data structures LIKE C", whereupon I had to lie down for half an hour and wash my mouth out with caustic soda. You might, however, like to learn Modula-2 and write it in that.